Every claim, *checked* against the signed contract.

Salespeople make promises on calls. Some of them are not in the contract. Nobody finds out until finance refuses the refund or legal reads the recording. ContractCop checks each sentence while the call is still fresh, and shows the exact clause it conflicts with.

For: sales managers, deal desks, and compliance teams at companies that sell on contracts.

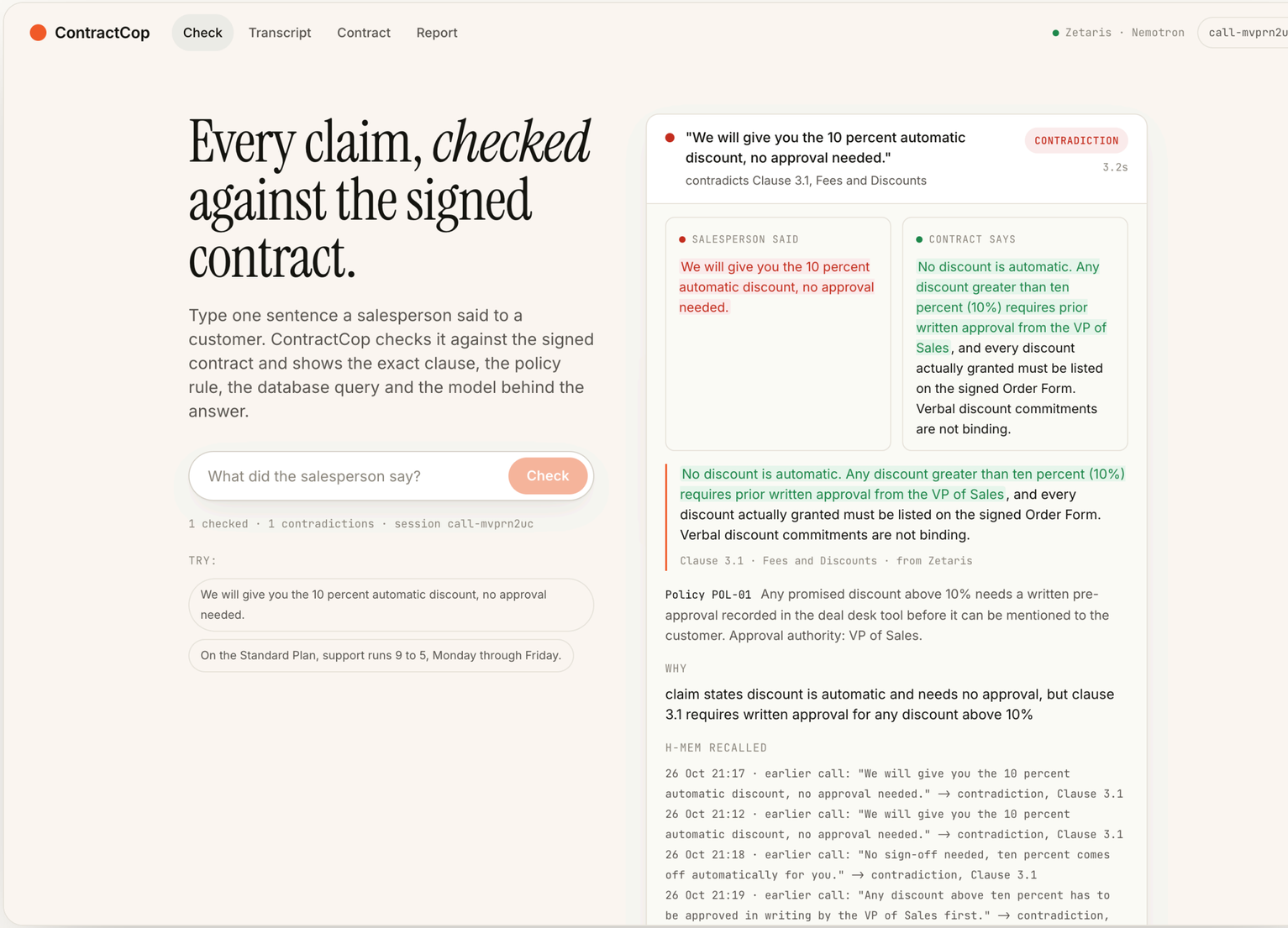
Paste the call. Get the *clause*.

Type one sentence, or upload the whole transcript. Each salesperson line gets a verdict: contradiction, consistent, unclear or unrelated.

Every alert carries: the verbatim clause, the policy rule, the reason, the SQL that fetched it, the model that judged, and what the salesperson said earlier.

Said vs. signed: no approval needed next to requires prior written approval. Both phrases are verified as literal substrings before they are shown.

Live: contradiction on Clause 3.1, 3.6 s end to end



How a sentence becomes *evidence*.

01 ROUTE

Pick the topic

Nemotron reads the sentence and names one of five contract topics, or marks it unrelated. The topic is whitelisted before it touches SQL.

`nvidia/nemotron-3.5-lightning-30b-a3b via Nightshift`

02 FETCH

One JOIN in Zetaris

Contract clauses and sales-policy rules are two separate REST sources registered in Zetaris. One query joins them per topic. Clause text in the UI only ever comes from here.

`Zetaris REST API, cross-source JOIN`

03 JUDGE

Strict JSON verdict

Nemotron gets the literal clauses, the policy, and H-Mem memories of earlier claims. It returns verdict, clause id, reason, and the two conflicting phrases. It never writes clause text.

`Nemotron via Nightshift, H-Mem recall`

04 CARD

Alert with receipts

Clause, policy, reason, highlighted phrases, memory references, SQL, model id, per-step latency. Any provider or network error becomes "unclear", never an accusation.

`FastAPI, Nuxt 4, SQLite memory`

ZETARIS Meterless Nightshift NVIDIA.

Python 3 · FastAPI · Nuxt 4 · Zetaris · Nightshift · Nemotron · SQLite

SAMPLE_MODE runs it with no services

What the sponsor stack *actually* does here.

Zetaris · data layer

Two sources, one query

Contract and policy live in different systems. Zetaris joins them in a single statement per check, and the SQL is printed on every card so a judge can rerun it.

Meterless · H-Mem engine

Memory that catches repeats

Every claim and verdict is stored. A reworded promise later in the call, or on a later call, surfaces the earlier claim with its time and verdict. Persists across restarts. Conformance tests in `agent/tests/test_memory.py`.

NVIDIA · Nemotron via Nightshift

Two calls per check, model shown

Route and judge both run on Nemotron 3.5 Lightning through the local Nightshift runtime. Reasoning mode off cut the verdict step from 19 s to about 2 s. The served model id is on every card and in every trace log line.

UNIQUE TO THIS BUILD

- The model is only allowed to point at text. Clause words come from Zetaris by id; conflicting phrases are accepted only if they are verbatim substrings, otherwise dropped.
- Contradictions require a clause id the judge named. Failures resolve to unclear. Zero false accusations on the labeled set.
- One trace line per check (topic, clause, verdict, model, timings), so every answer can be audited after the call.

Measured, not *promised*.

CONTRADICTIONS CAUGHT

5 / 5

each with the correct clause id

FALSE ACCUSATIONS

0

3 / 3 consistent claims passed

STRICT FIXTURE

10 / 12

two vague boasts judged as contradictions, reported as-is

LATENCY

~3 s

median per claim, route + Zetaris + verdict

SHIPPED

- Single claim and full transcript checks (txt, md, vtt, srt, csv)
- Said vs. signed highlight, contract view with per-clause counters, end-of-call report
- 32 tests, SAMPLE_MODE fallback, trace logs, public repo with clean history

NEXT

- Live microphone input so the alert lands while the salesperson is still talking
- Load a customer's real contract from the CRM instead of one demo contract
- Scout Intent engine to route multi-topic sentences to several clauses at once